

Timestamp Search for Lecture Video: A Multi-Cloud, Cloud-Native Design

Project Proposal (Milestone 1) • 6030 Cloud Computing • October 7, 2026

Joshua Jubenville
Dept. of Computer Science
[University]

[Teammate Name]
Dept. of Computer Science
[University]

[Teammate Name]
Dept. of Computer Science
[University]

Abstract—We propose a multi-cloud, cloud-native service for higher education that lets students search the spoken content of YouTube lecture videos and jump directly to the moment a topic is discussed. A query such as “dynamic programming” returns ranked links of the form `youtube.com/watch?v=ID&t=754s`, across one video or a whole course. YouTube keeps hosting and playback; our system acquires or generates transcripts, indexes them, and serves search. Containerized services run on Google Kubernetes Engine (GKE) with a PostgreSQL data tier on Google Cloud, while the React front end and AI services run on Microsoft Azure. GitHub Actions provides CI/CD and Terraform provisions both clouds. Each major design choice is recorded as an architecture decision record and backed by measurements from the running system.

Index Terms—cloud-native, multi-cloud, Kubernetes, semantic search, speech-to-text, higher education.

I. PROBLEM DEFINITION

Lecture video is one of the largest educational resources on the internet, and most of it is hosted on YouTube. It is also hard to search. A student reviewing for an exam knows a topic was covered somewhere in a semester of 75-minute lectures, but YouTube’s search finds videos by title and description, not by what is said inside them.

The result is scrubbing: dragging the playhead, guessing, and rewatching. Many videos already have captions that contain the answer, and those that do not can be transcribed automatically. What is missing is a system that indexes that text with timestamps and turns a question into a link to the right moment.

The project falls in the *Higher Education* domain. It is also a natural cloud problem: ingestion is bursty and compute-heavy, the index grows with the corpus, and queries need low latency at unpredictable volume. Each of these pulls the design toward a different set of services.

II. PROPOSED SYSTEM

The system has two roles. *Instructors* submit videos to index, either a single video or a playlist, typically a course’s lectures. *Students* search the indexed corpus by keyword or phrase through a web front end and receive a ranked list of timestamped links, each with a snippet of the surrounding transcript. Fig. 1 shows the architecture; its components are:

- 1) **Front end:** a React (Next.js) application on Azure Static Web Apps [1], which also handles sign-in.

- 2) **Query API:** a containerized service on GKE that ranks transcript chunks and returns timestamp links. It also publishes new indexing jobs to a Pub/Sub queue.
- 3) **Ingestion workers:** containers on GKE that pull jobs from the queue and scale with its depth. A source adapter fetches each video’s existing transcript, or its audio when none exists.
- 4) **Transcription:** runs only when no transcript exists, through a managed speech-to-text service.
- 5) **Chunking and embedding:** splits each transcript into overlapping time windows (for example, 30 s sliding by 15 s), tags each with its video ID and start time, and computes a vector embedding for semantic search.
- 6) **Data tier:** Cloud SQL for PostgreSQL holds video metadata, job state and chunks, using full-text search for keywords and the pgvector extension [2] for semantic search.

Ingestion runs through the queue in the background, so the query path reads only the database and search stays fast while new videos are processed. The starting corpus is a small, curated set of publicly available lecture videos that includes both captioned and uncaptioned videos, so both ingestion paths are exercised. The cost model (Section VI) covers growth to a much larger corpus.

III. TECHNOLOGY SERVICES

Table I maps each required layer to the service we will use and the reason for choosing it.

IV. CLOUD PROVIDER SELECTION RATIONALE

Google Cloud hosts the core and Azure hosts the front end and AI services. The split follows one rule: *keep the latency-sensitive path (query API to database) inside one cloud, and cross clouds only where calls are asynchronous or infrequent*.

Google Cloud runs the containers and data because GKE Autopilot’s per-pod billing fits a workload that is idle between ingestion bursts, and Cloud SQL for PostgreSQL supports pgvector, so one managed database covers all three kinds of search data. Keeping them together means a student’s query never leaves Google’s network.

Azure provides the front end and AI services. Static Web Apps includes authentication and a free tier, and Azure OpenAI and Azure AI Speech are mature managed AI services. Calls

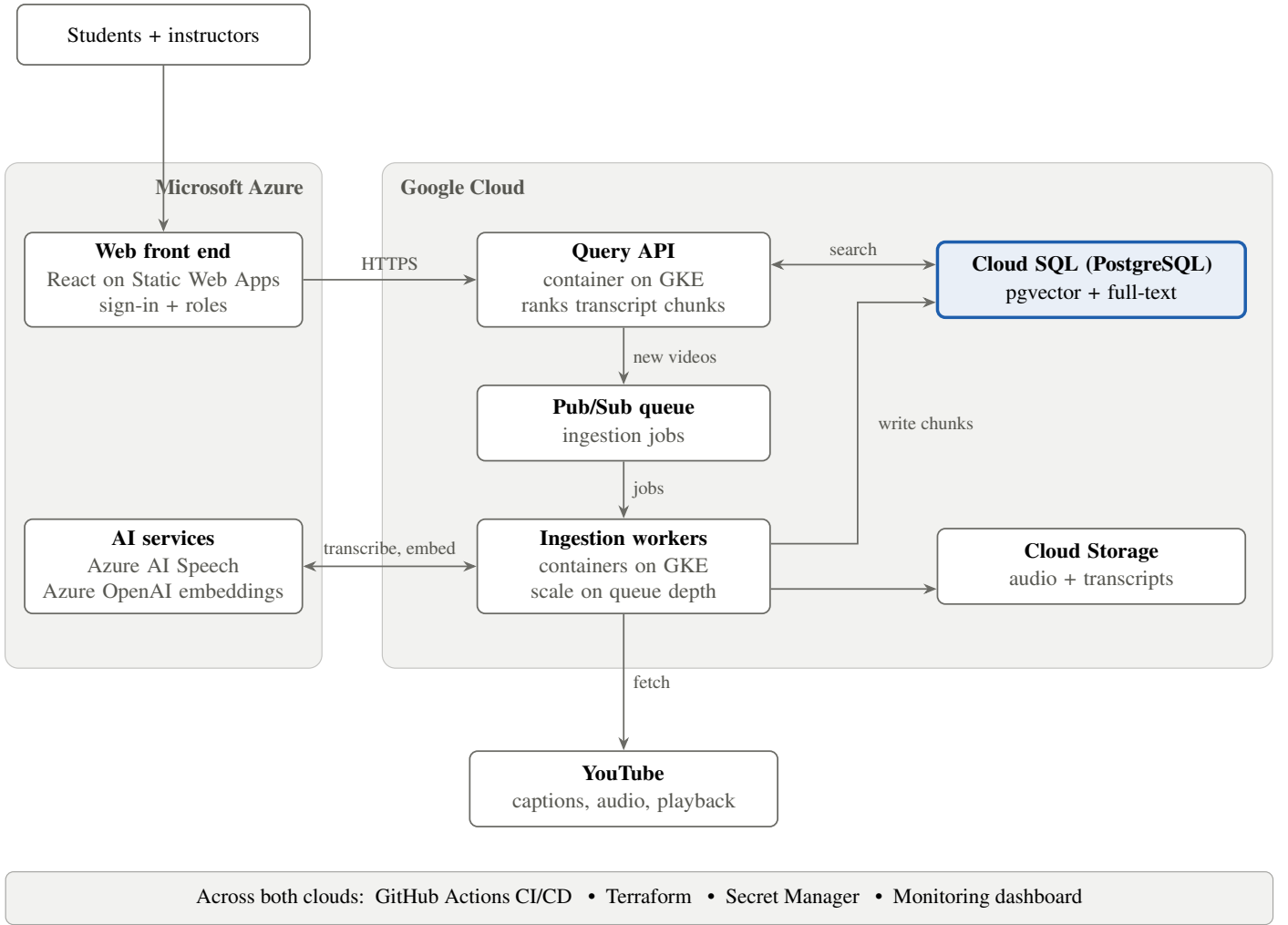


Fig. 1. System architecture. Containers and data run on Google Cloud; the front end and AI services run on Azure. The latency-sensitive path (query API to database) stays inside one cloud, while cross-cloud calls occur only during background ingestion.

to them happen during background ingestion, where an extra cross-cloud hop costs seconds on a minutes-long job rather than milliseconds on a query.

We considered placing the database on a second cloud, as in the course’s Example Architecture A, and rejected it because every search would cross clouds. We will measure that penalty to support the decision.

V. SECURITY

A. Identity Management

Students and instructors sign in through the front end’s built-in authentication (Microsoft Entra ID). Services use workload identities rather than stored keys: GKE Workload Identity for Google services, and federated credentials for calls to Azure where supported. GitHub Actions authenticates to both clouds through OpenID Connect, so no long-lived cloud keys are stored in the repository.

B. Access Control

The application has two roles: instructors can submit and remove videos, and students can search. In the cloud, each service receives a least-privilege role: the query API can read the database but not alter its schema, and ingestion workers can write chunks but cannot change IAM policy. Cloud SQL is reachable only on a private IP.

C. Secrets Management

Remaining secrets, such as the database password and any Azure API keys where federation is unavailable, live in Secret Manager and are mounted into pods at runtime.

VI. KEY DESIGN DECISIONS

Each major choice is documented as an Architecture Decision Record (ADR) [4]: the context and constraints, two to four real alternatives, the criteria, the decision and its consequences, and the evidence behind it. Because the build is largely agent-driven and fast, we can implement competing options for the most important decisions and measure them rather than argue from

Table I
TECHNOLOGY SERVICES BY LAYER

Layer	Service	Provider	Rationale
Front end	React (Next.js) on Azure Static Web Apps	Azure	Global hosting for a mostly static app, built-in authentication, deploys from GitHub Actions
Application tier	Docker containers on GKE Autopilot	Google Cloud	Required Kubernetes tier; Autopilot bills per running pod rather than per node, which suits bursty ingestion [3]
Queue	Pub/Sub	Google Cloud	Decouples submission from processing, with retries and dead-lettering for failed jobs
Data tier	Cloud SQL for PostgreSQL with pgvector	Google Cloud	One relational database serves metadata, keyword (full-text) and vector search
Object storage	Cloud Storage	Google Cloud	Temporary audio files and raw transcripts
Transcription	Azure AI Speech or Google Speech-to-Text	Azure / Google	Chosen by a measured comparison (Section VI)
Embeddings	Azure OpenAI embedding model	Azure	Managed embeddings for semantic search
CI/CD	GitHub Actions	GitHub	Builds images, runs tests, applies Terraform, deploys to GKE and Static Web Apps
Infrastructure as code	Terraform	Both clouds	Both providers in one codebase; full teardown and redeploy
Monitoring	Cloud Monitoring dashboard	Google Cloud	Queue depth, ingestion time, query latency and error rate
Secrets	Secret Manager	Google Cloud	Runtime secrets mounted into pods, never stored in code or images

Table II
ARCHITECTURE DECISIONS TO BE RECORDED AND DEFENDED

Decision	Options considered	Deciding criteria	Evidence we will produce
Provider split	Core on one cloud with front end and AI on another; database on a second cloud (Example A); best service per provider	Query latency, egress cost, operational complexity	Latency of a same-cloud vs. cross-cloud database query
Transcript source	YouTube captions; Azure AI Speech; Google Speech-to-Text	Cost per audio hour, word error rate, timestamp precision	The same lectures through each service, compared
Data tier	PostgreSQL with pgvector; NoSQL (Firestore or Cosmos DB) plus a separate search service	Search capability, idle cost, number of moving parts	Feature fit and monthly cost of each
Search approach	Keyword (full-text); semantic (vector); hybrid	Result quality on our test set, cost per query	Quality scores for each approach
Chunking	Caption lines; fixed windows; overlapping windows	Recall on phrases spanning boundaries, index size	Recall and index size at several window sizes
Ingestion scaling	Fixed worker pool; autoscaling on queue depth; one Kubernetes Job per video	Throughput, idle cost, recovery from a failed job	Burst test: a full course playlist submitted at once
Cost model	—	Cost to ingest one video, cost per query, fixed monthly cost	Projected monthly cost at 10, 10 ⁴ and 10 ⁶ videos

documentation alone. Table II lists the decisions. The provider split, transcript source and cost model carry the most weight; the cost model feeds directly into the final cost analysis.

VII. ASSUMPTIONS AND CONSTRAINTS

We assume *authorized programmatic access to YouTube transcripts and audio*. Today, the official YouTube Data API lets an application download caption tracks only for videos it owns [5], and YouTube’s terms restrict downloading video content. A production version would need a partnership with YouTube or permission from rights holders. We name this assumption deliberately: integrating with the platform where most lecture video is hosted is what makes the system useful.

To contain the assumption, all YouTube access sits behind a single *source adapter* with a narrow interface: get a video’s timestamped transcript, or get its audio. Nothing downstream depends on YouTube. If the assumption fails, one component is replaced, for example by an adapter that ingests openly licensed

copies from the original publisher (such as MIT OpenCourseWare) while results still link to YouTube for playback.

The project also runs on student cloud accounts with limited budgets, so idle cost matters as much as cost at scale. All content is publicly available lecture material used for educational purposes.

VIII. METHODOLOGY AND EVALUATION

The team designs the system and writes each ADR; implementation is done largely with AI coding agents, and every member contributes to every decision rather than owning a single layer. We will keep a running log of how AI tools were used, where they helped, and where their output had to be corrected. That log becomes the AI-usage reflection in the final presentation. We will evaluate the system on four axes:

- **Search quality:** a test set of queries with known correct timestamps, labeled by the team from the starter corpus. A

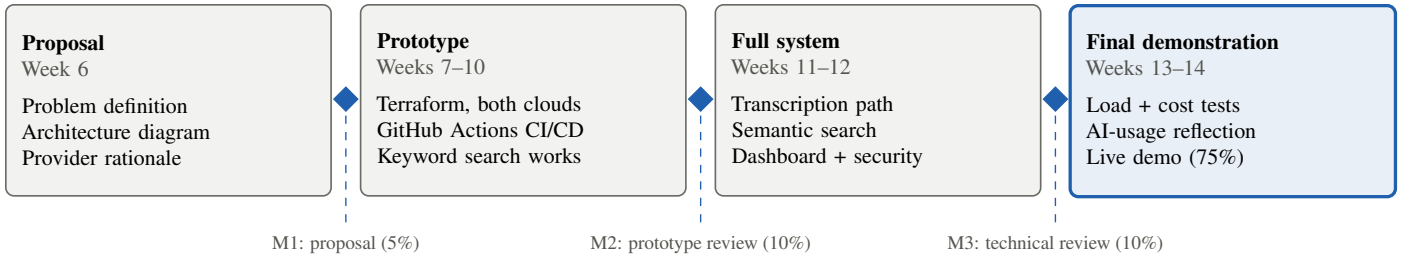


Fig. 2. Project roadmap aligned to the course milestones. Diamonds mark the graded reviews at the end of weeks 6, 10 and 12; the final demonstration in week 14 carries 75% of the project grade.

Table III
RISKS AND MITIGATIONS

Risk	Mitigation
YouTube access unavailable in practice	The source adapter isolates it; fall back to a publisher-source adapter or recorded test data
Cloud spend exceeds student budgets	Budget alerts on both clouds; workers scale to zero; tear down idle infrastructure with Terraform, since the database and cluster bill while running
Cross-cloud latency or egress cost	Keep the query path in one cloud; measure the cross-cloud calls that remain
Poor transcription of technical vocabulary	Compare services on our own lectures; prefer existing human captions
Semantic search adds cost without better results	Measure against keyword search on the test set; keep it only if justified
Agent-written code hides design flaws	Trace every component to an ADR; review Terraform plans in pull requests before they are applied

result is correct if it lands within a set tolerance of the true moment; we report how often the right moment appears in the top results for keyword, semantic and hybrid search.

- **Performance:** query latency (median and 95th percentile) under simulated load, and end-to-end ingestion time per hour of video.
- **Cost:** measured cost per ingested video and per query, plus fixed monthly costs, extrapolated to the scale projections in Table II.
- **Operations:** a monitoring dashboard showing queue depth, ingestion throughput, query latency and error rate, used during load tests and the live demonstration.

IX. TIMELINE AND DELIVERABLES

Work follows the course’s graded milestones (Fig. 2). Each phase ends with working software and the ADRs it settles. The final deliverables in week 14 are a live system demonstration, an architecture presentation built on the ADRs, a cost analysis from measured costs and scale projections, and an AI-usage reflection with lessons learned.

X. RISKS AND EXPECTED OUTCOMES

Table III lists the main risks and how we will mitigate them.

We expect to deliver a working search service over a curated lecture corpus and, more importantly, a defensible account of

why it is built the way it is: which services fit each part of the problem, what each choice costs, and what would change first as the corpus grows from ten videos to a million.

REFERENCES

- [1] Microsoft, “What is Azure Static Web Apps?” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/azure/static-web-apps/overview>
- [2] pgvector: Open-source vector similarity search for Postgres. [Online]. Available: <https://github.com/pgvector/pgvector>
- [3] Google Cloud, “Autopilot overview,” Google Kubernetes Engine documentation. [Online]. Available: <https://cloud.google.com/kubernetes-engine/docs/concepts/autopilot-overview>
- [4] M. Nygard, “Documenting architecture decisions,” Cognitect blog, Nov. 2011. [Online]. Available: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- [5] Google, “Captions,” YouTube Data API v3 reference. [Online]. Available: <https://developers.google.com/youtube/v3/docs/captions>